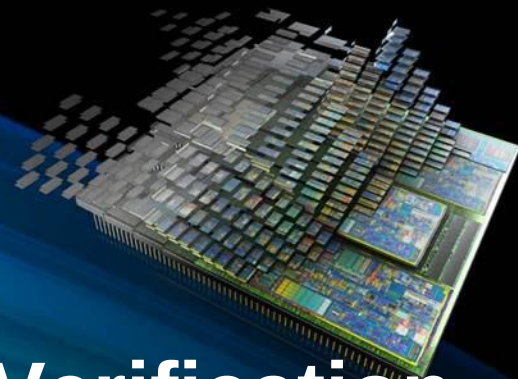


Verification Academy



SystemVerilog OOP for UVM Verification

Inheritance & Polymorphism

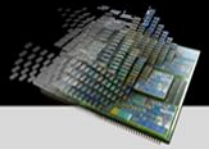
Dave Rich

Verification Architect

info@verificationacademy.com | www.verificationacademy.com

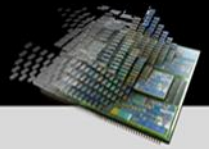


Extending SystemVerilog Classes

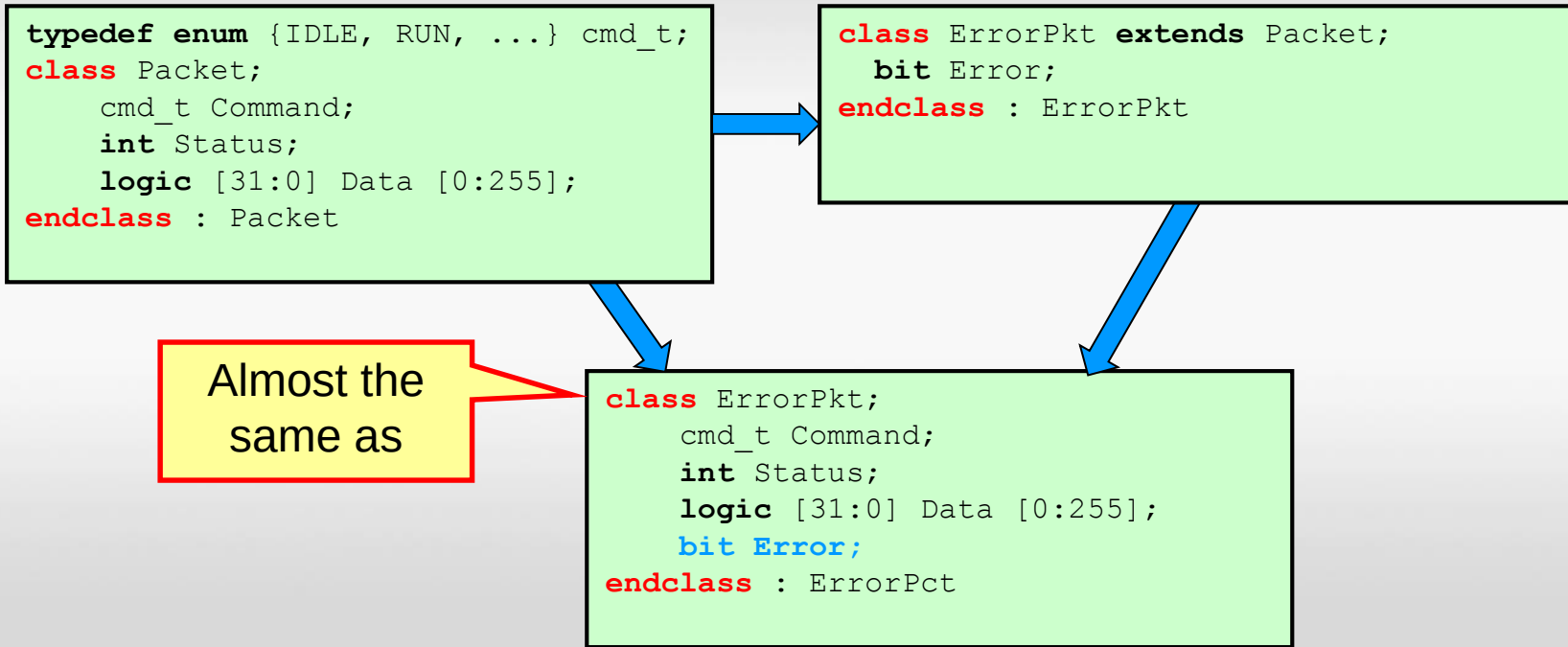


- Inherit or extend properties and methods from other classes
 - Original class is the *base* or *super* class
 - Extended class is the *derived* or *sub*-class
- Allows customization without breaking or rewriting known-good functionality in the base class

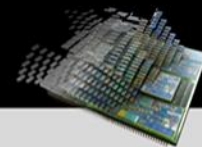
Extending Class Properties



- Inheritance creates a new class type based on an existing class type



Extending Class Methods



- Add new method or add new behavior of an existing method

```
typedef enum {IDLE, RUN, ...} cmd_t;
class Packet;
    cmd_t Command;
    int Status;
    logic [31:0] Data [0:255];
    function void SetStatus (input int a);
        Status = a;
    endfunction
endclass : Packet
```

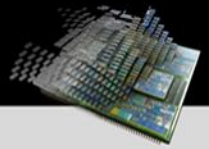


```
class ErrorPkt extends Packet;
    bit Error;
    function bit ShowError();
        return(Error);
    endfunction

    function void SetStatus (input int a);
        if (Error)
            super.SetStatus(a+1);
        else
            super.SetStatus(a);
    endfunction
```

super. looks back from
the base class

Constructing Extended Classes



```
class thing;  
  string name;  
  function new;  
    name = "thing";  
  endfunction  
endclass
```

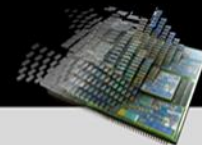
```
class component extends thing;  
  int id;  
  function new;  
    super.new(),  
    id = 1;  
  endfunction  
endclass
```

call to base
constructor must
be first
statement

```
class thing;  
  string m_name;  
  function new( string name );  
    m_name = name;  
  endfunction  
endclass  
class component extends thing;  
  int id;  
  function new;  
    super.new( "component" );  
    id = 1;  
  endfunction  
endclass
```

need explicit
call to new()
with arguments

Class Variables with Inheritance

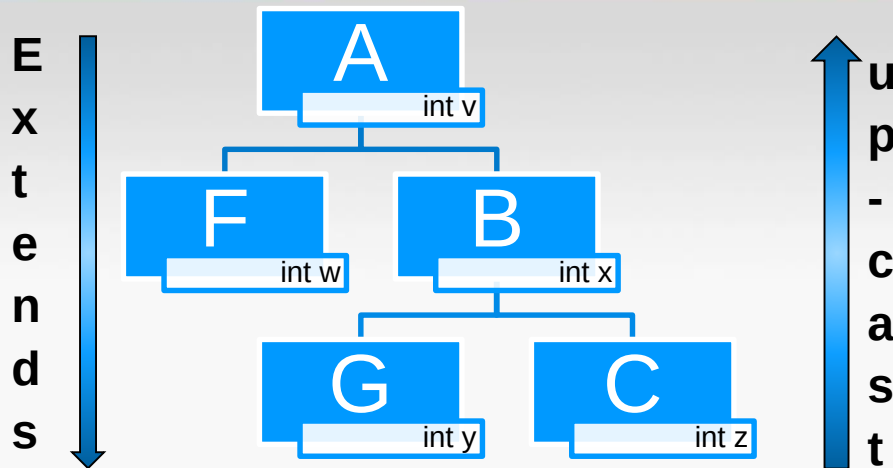
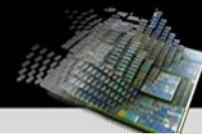


- **Base class variable can hold handles to extended objects, but not the reverse**

```
class Packet;
    cmd_t Command;
    int Status;
    logic [31:0] Data [0:255];
    function void SetStatus (input int a);
        Status = a;
    endfunction
endclass : Packet
class ErrorPkt extends Packet;
    bit Error;
    function bit ShowError();
        return(Error);
    endfunction
    function void SetStatus (input int a);
        Status = a + 1;
    endfunction
endclass : ErrorPkt
```

```
begin
    Packet Pkt_h;
    ErrorPkt Epkt_h;
    Epkt_h = new;
    Epkt_h.Command = RUN;
    Epkt_h.Error = 1;
    Epkt_h.SetStatus(1); // Status == 2
    Pkt_h = Epkt_h;
    Pkt_h.Command = IDLE;
    Pkt_h.SetStatus(1); // Status == 1
    Pkt_h.Error = 0; // compiler error
    Pkt_h = new;
    Epkt_h = Pkt_h; // compiler error
                    // Epkt_h.Error would not exist
```

Dynamic Casting of Class handles

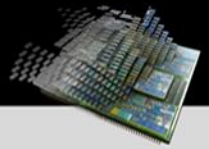


class type	available vars
A	v
F extends A	v,w
B extends A	v,x
G extends B	v,x,y
C extends B	v,x,z

```
function void func(A a_h);
    G g_h; C c_h;
    $display(a_h.v);
    if ( $cast(c_h, a_h) )
        $display(c_h.z);
    else if ( $cast(g_h, a_h) )
        $display(g_h.y);
endfunction
```

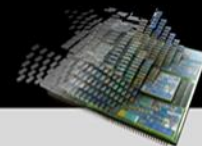
- up-casting is always legal via direct assignment
- down-casting requires a run-time check using `$cast`

Polymorphism



- Ability to have the same code act differently for different types
- A key requirement for any OOP Language
- Statically at compile time by using parameterized types
- Dynamically during run-time using virtual class methods

Virtual Class Methods



- Chose the method by the object type, not the class variable

```
typedef enum {IDLE, RUN, ...} cmd_t;
class Packet;

    cmd_t Command;
    int Status;
    logic [31:0] Data [0:255];
    virtual function void SetStatus
        (input int a);

    Status = a;
endfunction
endclass : Packet
```



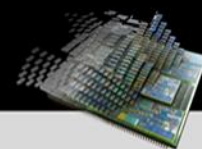
```
class ErrorPkt extends Packet;
    bit Error;
    virtual function void SetStatus
        (input int a);

        Status = a + 1;
    endfunction
endclass : ErrorPkt
```

```
task run(Packet p_h);
    ...
    p_h.SetStatus(5);
    $display(p_h.status); // 5 or 6
    ...
endtask
```

- Override must have the same prototype
- Once **virtual** -> Always **virtual**

Mixing Virtual and Non-Virtual Methods



- Clone is construction with copy

```
class A;  
  int p1;  
  virtual function A clone();  
    A h = new;  
    h.copy(this);  
    return h;  
endfunction  
function void copy(A rhs);  
  this.p1 = rhs.p1;  
endfunction  
endclass
```

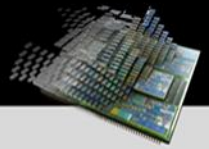
```
class B extends A;  
  int p2;  
  virtual function A clone();  
    B h = new;  
    h.copy(this);  
    return h;  
endfunction  
function void copy(B rhs);  
  this.p2 = rhs.p2;  
  super.copy(rhs);  
endfunction  
endclass
```

```
class C extends B;  
  int p3;  
  virtual function A clone();  
    C h = new;  
    h.copy(this);  
    return h;  
endfunction  
function void copy(C rhs);  
  this.p3 = rhs.p3;  
  super.copy(rhs);  
endfunction  
endclass
```

```
task burst(A a_h, int N);  
  repeat (N)  
    send( a_h.clone() );  
endtask  
task send(A a_h);  
  ...  
endtask
```

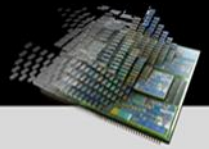
Each call to send() gets a new object that is a full copy of the object passed into burst()

Accessibility and Abstract Classes



- By default, all members of a class are publically visible
 - If you have a handle to an object, you can access any member
 - **local** qualifier restricts access to only within the declared class
 - **protected** qualifier restricts access to only within the declared class and an extension of that class
- Abstract class can never be constructed directly
 - Can construct class extended from abstract class
 - Provide common API for class methodology
 - Can declare virtual method prototypes that must be overridden in extended class

Abstract Class example



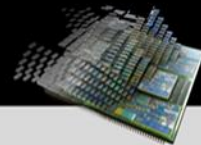
```
virtual class Component;  
  local Component children[$];  
  protected string m_name;  
  function new (string name,  
               Component parent);  
    m_name = name;  
    if (parent != null)  
      parent.children.push_back(this);  
  endfunction : new  
  pure virtual function void print();  
  function void printtree();  
    foreach (children[child])  
      children[child].printtree();  
    this.print();  
  endfunction : printtree  
endclass : Component
```

Abstract class declared
with ***virtual*** keyword

local limits access outside base class
protected limits access outside class

Since function is declared as pure
virtual, an implementation must be
provided in the extended class

Extending Abstract Class into Concrete

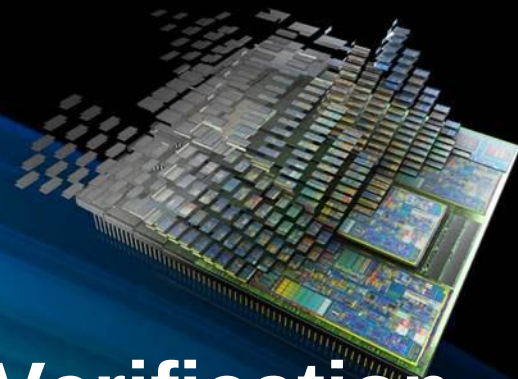


```
class MyComponent extends Component
  function new(string name,
              Component parent);
    super.new(name, parent);
endfunction : new
virtual function void print();
  $display(m_name);
endfunction
endclass : MyComponent
```

```
initial begin
  MyComponent m1,m2,m3,m4;
  m1 = new("roof",null);
  m2 = new("kitchen",m1);
  m3 = new("bedroom",m1);
  m4 = new("stove",m2);
  m1.printtree();
  m1.m_name = "off"; // illegal
  m1.print;
end
```

stove
kitchen
bedroom
roof

Verification Academy



SystemVerilog OOP for UVM Verification

Inheritance & Polymorphism

Dave Rich

Verification Architect

info@verificationacademy.com | www.verificationacademy.com

